

DOKUMENTACJA PROJEKTOWA MAS

System do zakupu gier

Wersja 1.0

Data: 18.05.2026

Autor: Ewelina Paczkowska

Numer indeksu: s30995

Spis treści

1.	Wymagania użytkownika	3
1.1	Kontekst i tło	3
1.2	Wymagania użytkownika	3
2.	Diagramy przypadków użycia	4
2.1	Aktorzy systemu	4
2.2	Diagram przypadków użycia – widok ogólny	5
3.	Wymagania niefunkcjonalne	6
3.1	Wydajność	6
3.2	Bezpieczeństwo	6
3.3	Niezawodność i dostępność	6
3.4	Skalowalność	6
3.5	Użyteczność	6
3.6	Przenośność i kompatybilność	6
4.	Diagram klas – analityczny	8
4.1	Diagram	8
5.	Diagram klas – projektowy	9
5.1	Diagram	9
6.	Scenariusz przypadku użycia	10
6.1	Identyfikacja przypadku użycia	10
6.2	Scenariusz główny	10
6.3	Scenariusze alternatywne	10
6.4	Scenariusze wyjątkowe	10
7.	Diagram aktywności dla przypadku użycia	11
7.1	Diagram	11
8.	Diagram stanu dla klasy	12
8.1	Wybrana klasa	12
8.2	Diagram	12
9.	Projekt GUI	13
9.1	Mapa nawigacji	13
9.2	Makiety ekranów	13
10.	Omówienie decyzji projektowych i skutków analizy dynamicznej	14
10.1	Kluczowe decyzje architektoniczne	14
10.2	Decyzje dotyczące modelu danych	14
10.3	Analiza dynamiczna – wnioski	15
10.4	Kompromisy i alternatywy	16
10.5	Wnioski końcowe	16

1. Wymagania użytkownika

1.1 Kontekst i tło

System jest samodzielną aplikacją służącą do zarządzania biblioteką gier komputerowych oraz relacjami pomiędzy graczami, grami i twórcami. Aplikacja została przygotowana w technologii obiektowej Java.

Celem systemu jest:

- przechowywanie informacji o grach,
- zarządzanie graczami,
- obsługa zakupów gier,
- obsługa recenzji gier,
- zarządzanie developerami,
- rozróżnienie gier wydanych i będących w produkcji.

1.2 Wymagania użytkownika

System przechowuje dane o grach dostępnych na platformie: nazwę, datę wydania, listę tagów, opcjonalny opis, maksymalny ram dla gry wyrażany w GB. Gra może posiadać wiele tagów, a musi posiadać co najmniej jeden.

W systemie są przechowywane dane o deweloperach i wydawcach. Deweloper odpowiada za tworzenie gier, natomiast wydawca jest odpowiedzialny za ich wydawanie. Jest także deweloper niezależny – taki, który odpowiada jednocześnie za tworzenie i wydawanie gier. Dla każdego dewelopera i wydawcy zapisywana jest nazwa.

Gra może być w trakcie produkcji lub wydana – w przypadku gry w trakcie produkcji przechowywana jest planowana data wydania gry, natomiast jeżeli gra została wydana w systemie jest informacja o dacie wydania gry. Stan gry zmienia się dynamicznie.

Istnieją różne typy gier: singleplayer i multiplayer. Gra może być zarówno jednoosobowa, jak i wieloosobowa. Dla gry jednoosobowej przechowywana jest informacja o nazwie profilu gracza, dla gry wieloosobowej przechowywana jest maksymalna liczba graczy.

W systemie są przechowywane informacje o graczach: nazwy graczy i listę posiadanych gier. Gracz może zakupić grę na platformie, dla każdego zamówienia zapamiętana jest data zamówienia.

Do gry mogą zostać dodane recenzje. Każda z recenzji zawiera treść. W przypadku usunięcia gry z systemu jej recenzje również zostają usunięte.

System powinien umożliwiać:

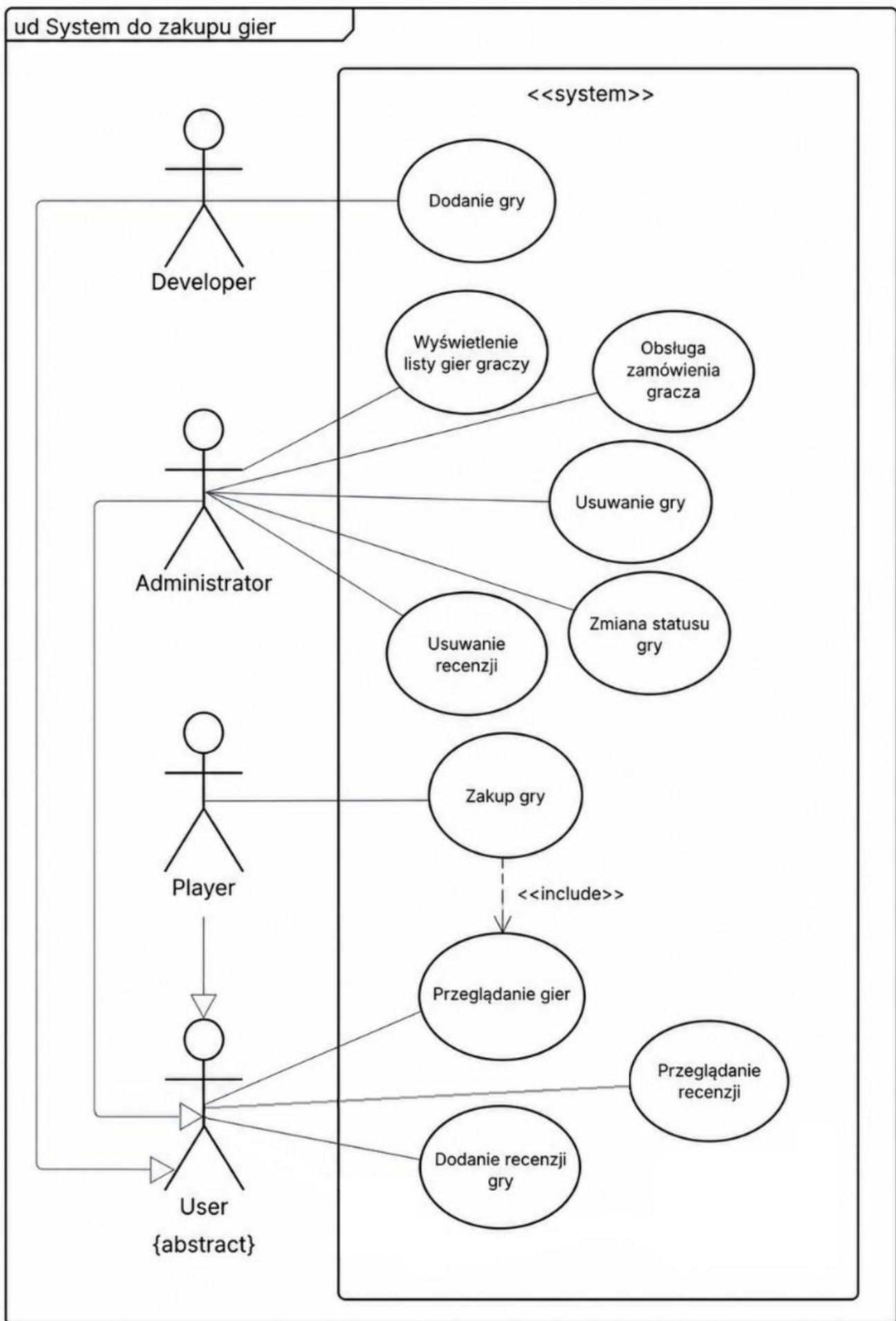
1. Przeglądanie gier w systemie
2. Dodanie nowej gry
3. Dodanie recenzji do gry
4. Wyświetlenie recenzji gry
5. Zakup gry
6. Usuwanie gier
7. Wyświetlanie zakupionych przez gracza gier
8. Zmianę statusu gry – w trakcie produkcji lub wydana
9. Dodanie nowego dewelopera lub gracza

2. Diagramy przypadków użycia

2.1 Aktorzy systemu

- Administrator: obsługa zamówienia gracza, usuwanie gry, przeglądanie gier, przeglądanie recenzji, dodanie recenzji gry, usuwanie recenzji, wyświetlenie listy gier graczy, zmiana statusu gry
- Deweloper: dodanie nowej gry, przeglądanie gier, przeglądanie recenzji, dodanie recenzji gry
- Gracz: przeglądanie gier, zakup gry (możliwe w trakcie przeglądania gier), dodanie recenzji gry, przeglądanie recenzji

2.2 Diagram przypadków użycia – widok ogólny



3. Wymagania niefunkcjonalne

3.1 Wydajność

- WN-P-01: System powinien umożliwiać szybkie dodawanie oraz usuwanie obiektów takich jak gry, recenzje oraz zakupy
- WN-P-02: Operacje wykonywane na kolekcjach obiektów powinny być realizowane w sposób optymalny, bez niepotrzebnego tworzenia kopii dużych zbiorów danych
- WN-P-03: Operacje zapisu i odczytu ekstensji powinny być wykonywane w akceptowalnym czasie dla rozmiaru danych używanego w projekcie

3.2 Bezpieczeństwo

- WN-B-01: System powinien walidować dane wejściowe – m.in. poprawność dat i nazw.
- WN-B-02: System powinien uniemożliwiać dodanie pustych tagów do gry.
- WN-B-03: System powinien uniemożliwiać ustawienie wymaganej pamięci RAM większej niż maksymalna obsługiwana wartość.

3.3 niezawodność i dostępność

- WN-N-01: System powinien zachowywać spójność danych podczas dodawania oraz usuwania relacji między obiektami.
- WN-N-02: Usunięcie gry powinno powodować poprawne usunięcie lub odłączenie powiązanych obiektów, takich jak recenzje, developerzy oraz zakupy.
- WN-N-03: Usunięcie zakupu powinno powodować usunięcie powiązania zarówno po stronie gracza, jak i gry.
- WN-N-04: System powinien umożliwiać zapis i odczyt ekstensji z pliku, aby dane mogły zostać odtworzone po ponownym uruchomieniu aplikacji.

3.4 Skalowalność

- WN-S-01: Struktura systemu powinna umożliwiać dodawanie nowych typów gier bez konieczności przebudowy całej aplikacji.
- WN-S-02: System powinien umożliwiać rozszerzenie modelu o kolejne klasy, np. osiągnięcia lub promocje.
- WN-S-03: System powinien umożliwiać rozwijanie istniejących relacji, np. dodanie większej liczby atrybutów do zakupu lub recenzji.

3.5 Użyteczność

- WN-U-01: System powinien być obsługiwany intuicyjnie bez uprzedniego szkolenia
- WN-U-02: Komunikaty wyjątków powinny wskazywać przyczynę błędu, np. pustą nazwę, brak obiektu lub niepoprawną datę.

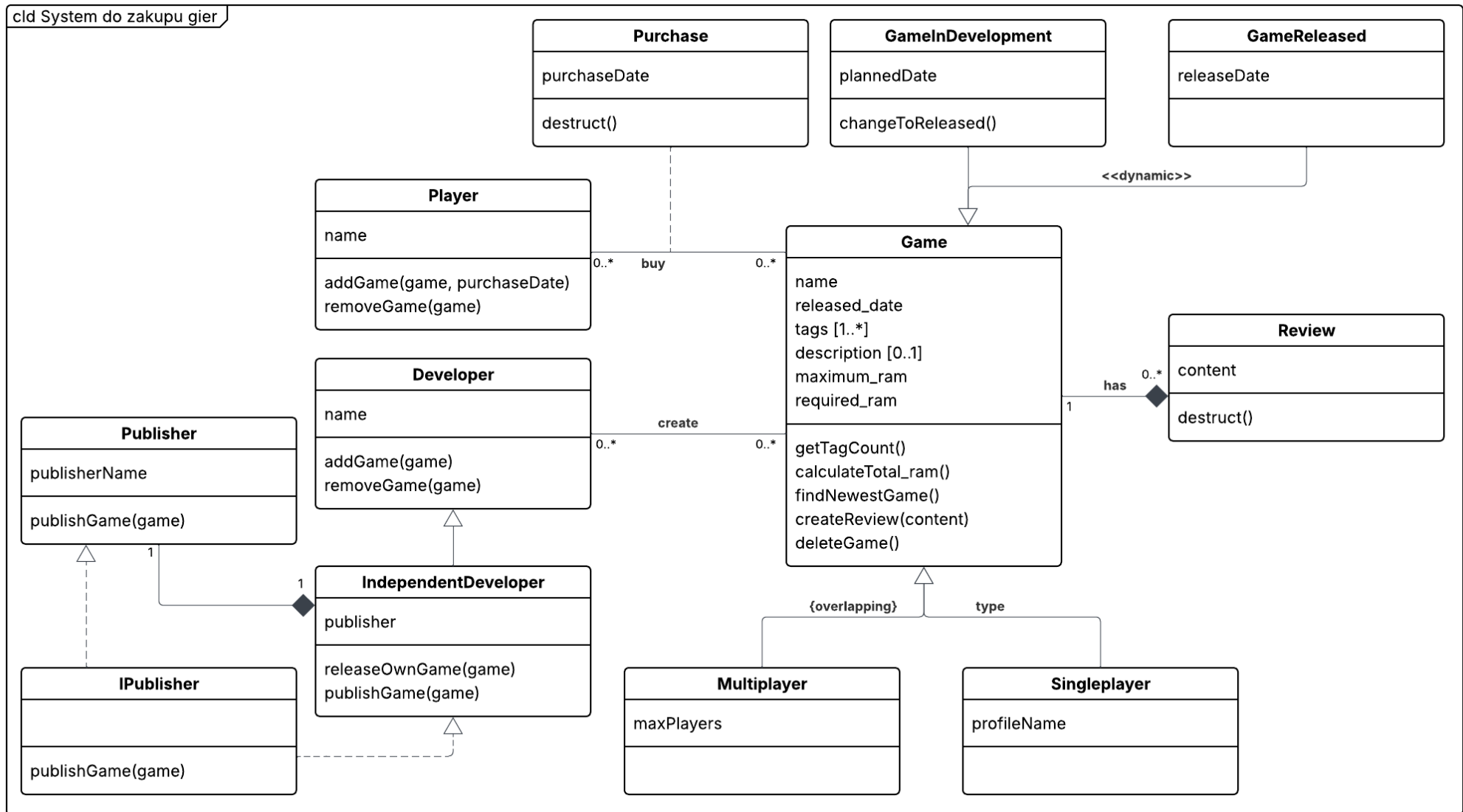
3.6 Przenośność i kompatybilność

- WN-K-01: System powinien działać w środowisku Java.
- WN-K-02: System powinien być niezależny od systemu operacyjnego.

- WN-K-03: System nie powinien wymagać zewnętrznych bibliotek do obsługi podstawowej logiki domenowej.
- WN-K-04: Dane zapisywane w ekstensji powinny być możliwe do odtworzenia w tym samym środowisku uruchomieniowym.
- WN-K-05: Kod powinien być możliwy do uruchomienia w standardowym środowisku IDE obsługującym Javę.

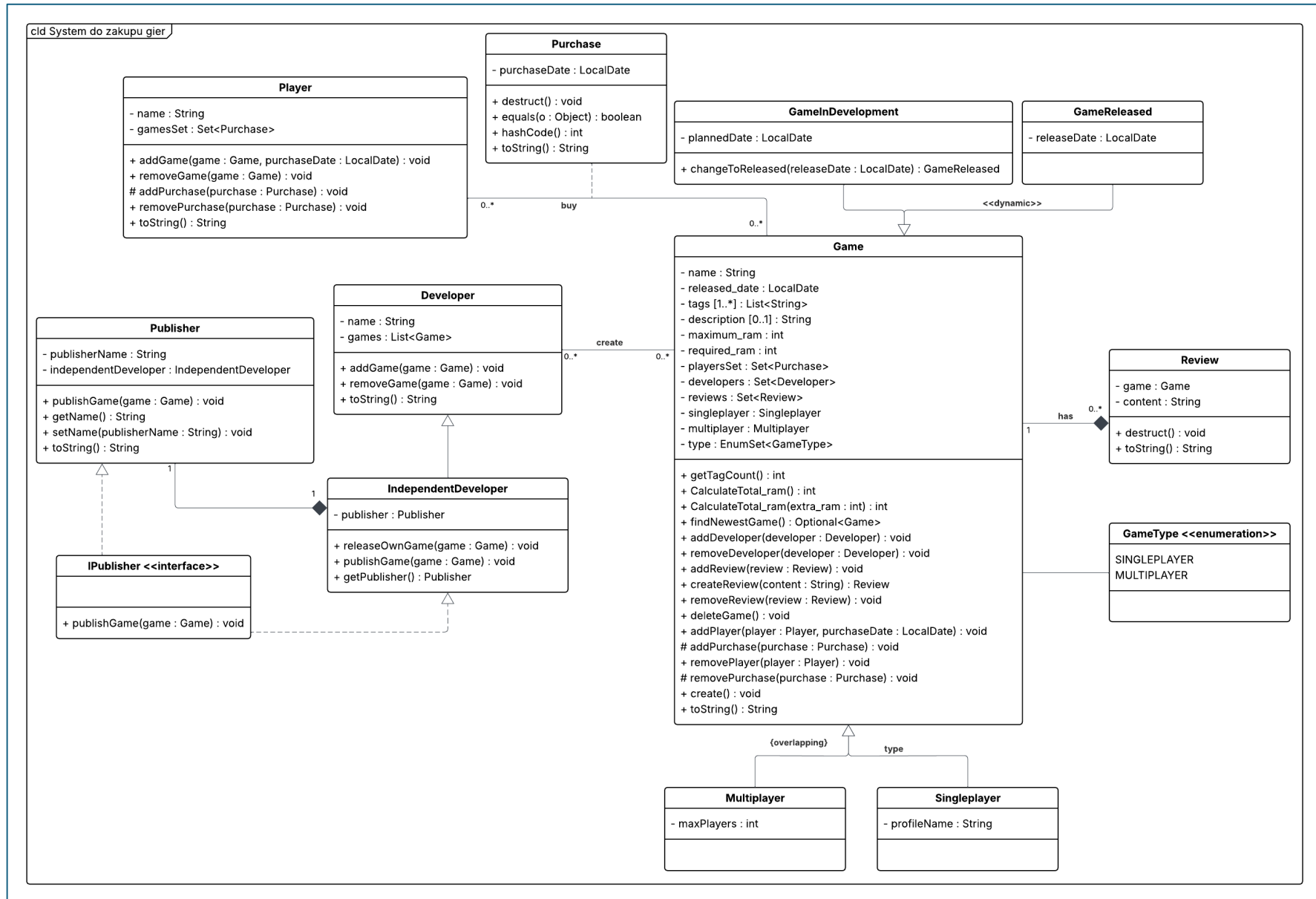
4. Diagram klas – analityczny

4.1 Diagram



5. Diagram klas – projektowy

5.1 Diagram



6. Scenariusz przypadku użycia

6.1 Identyfikacja przypadku użycia

Nazwa: Zakup gry

Aktor inicjujący: Gracz

Warunek wstępny: Gracz istnieje w systemie, a wybrana gra jest dostępna na platformie.

Warunek końcowy (sukces): Gra zostaje dodana do biblioteki gracza, a w systemie zostaje utworzony obiekt Purchase z datą zakupu.

Warunek końcowy (porażka): Gra nie zostaje dodana do biblioteki gracza, a zakup nie zostaje zapisany.

6.2 Scenariusz główny

Krok 1: Gracz wybiera grę w trakcie przeglądania listy gier, którą chce kupić.

Krok 2: System wyświetla informacje o grze.

Krok 3: Gracz potwierdza zakup gry.

Krok 4: System tworzy obiekt Purchase z datą zakupu.

Krok 5: System dodaje grę do biblioteki gracza.

6.3 Scenariusze alternatywne

Alt. 3a: Jeżeli gracz posiada już daną grę, system nie tworzy ponownego zakupu i informuje, że gra znajduje się już w bibliotece.

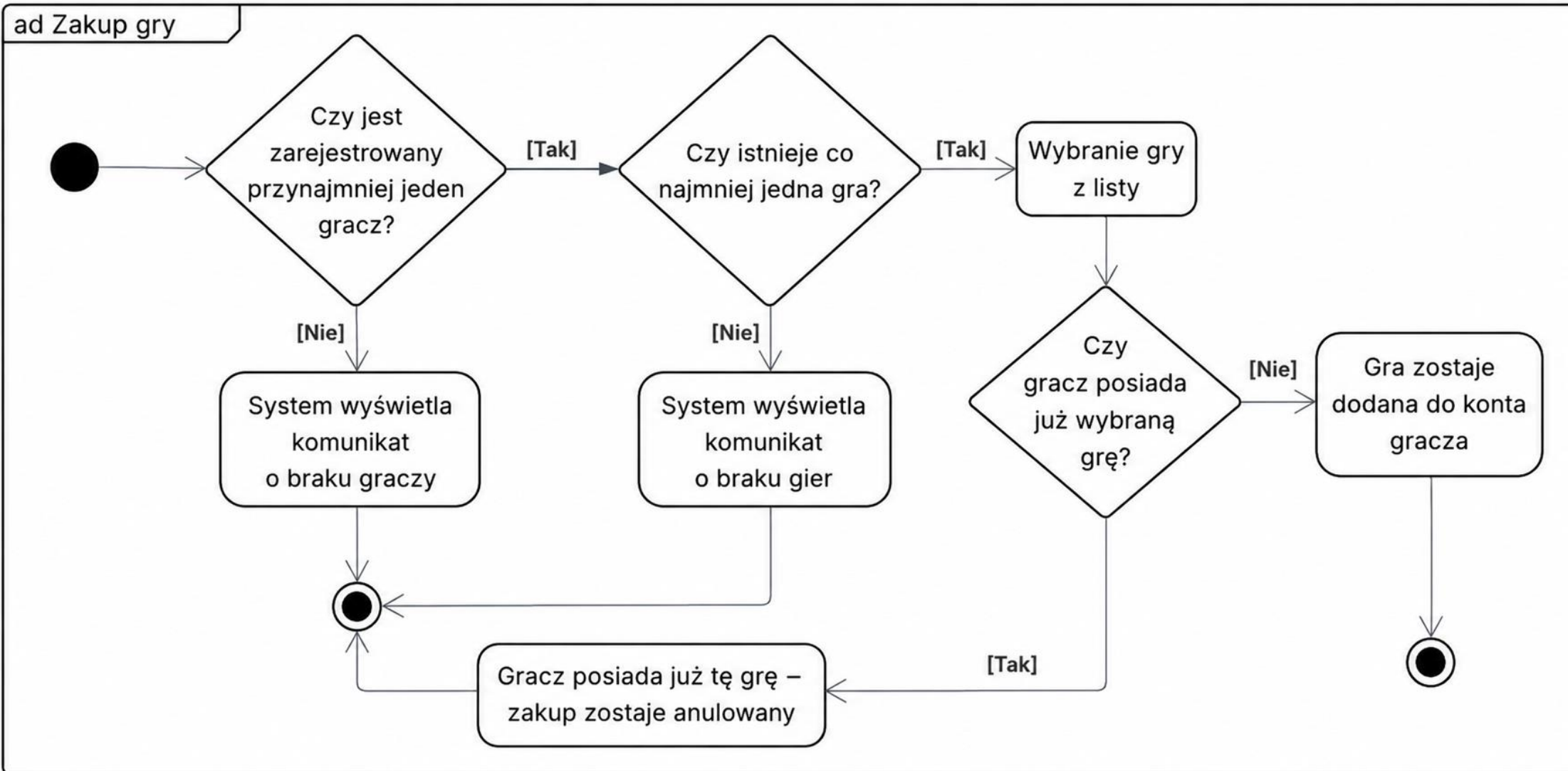
Alt. 3b: Jeżeli gracz rezygnuje z zakupu, system przerywa proces i nie zapisuje żadnych zmian.

6.4 Scenariusze wyjątkowe

Wyjątek 2a: Jeżeli nie ma gier w systemie, system wyświetli odpowiedni komunikat o braku gier.

7. Diagram aktywności dla przypadku użycia

7.1 Diagram



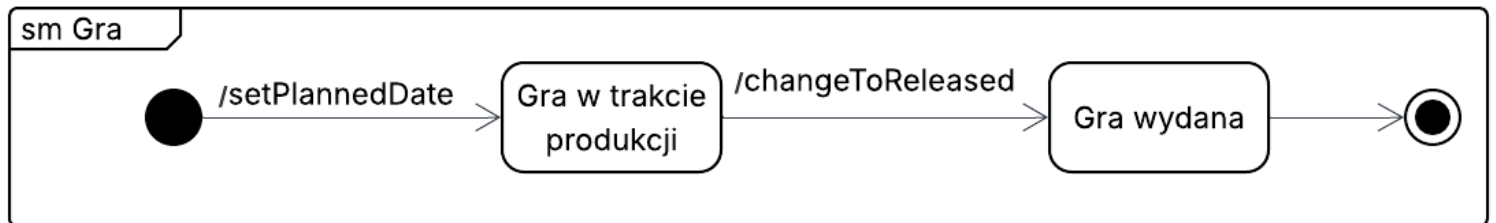
8. Diagram stanu dla klasy

8.1 Wybrana klasa

Wybrana klasa: Game

Do diagramu stanu wybrano klasę Game, ponieważ gra w systemie może znajdować się w różnych stanach – w trakcie produkcji i wydana. Początkowo gra może być w fazie tworzenia - GameInDevelopment, a potem przejść do stanu GameReleased – gry wydanej.

8.2 Diagram



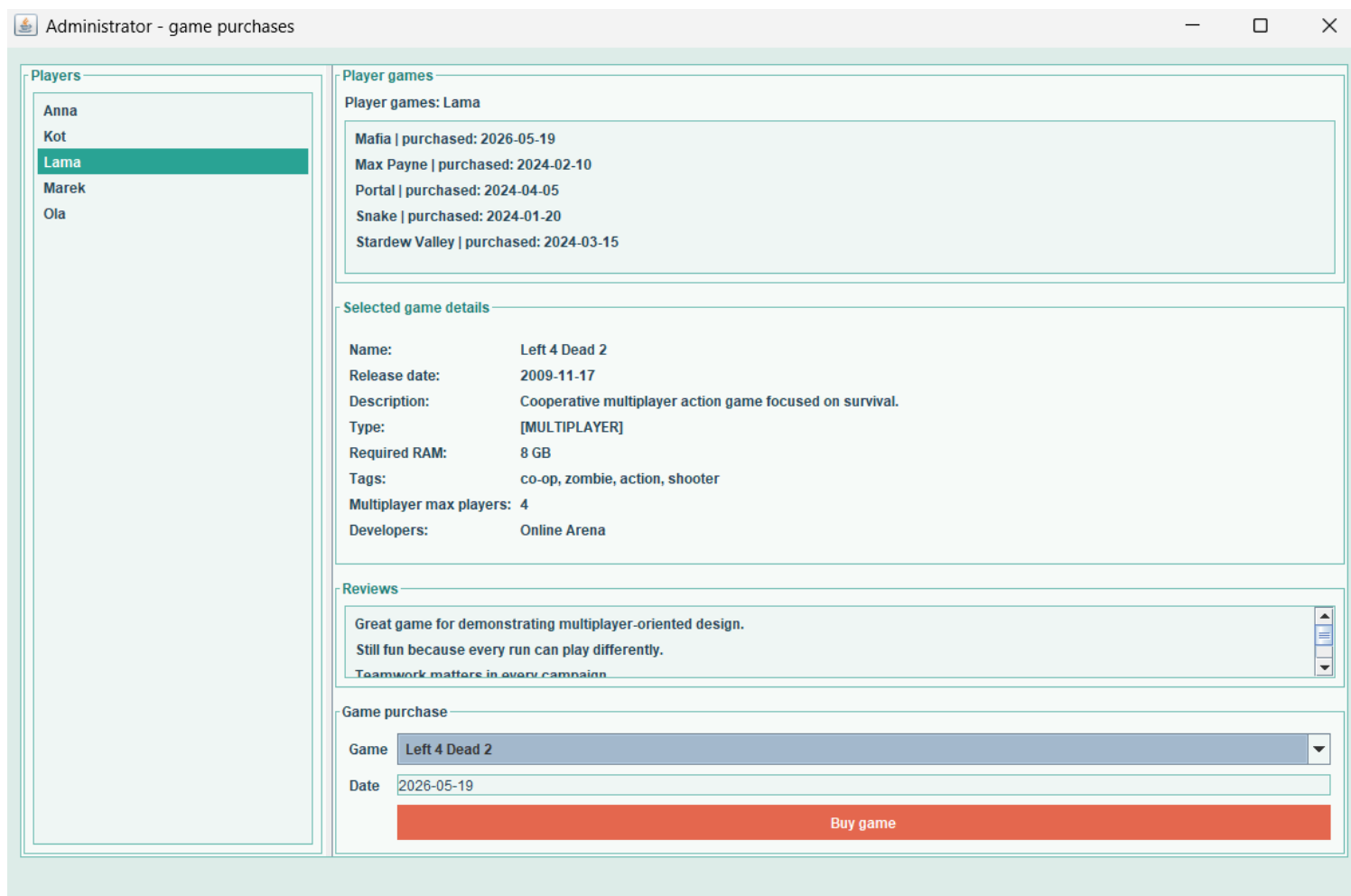
9. Projekt GUI

9.1 Mapa nawigacji

Po uruchomieniu systemu widoczny jest jeden główny ekran z perspektywy administratora. Po lewej stronie ekranu znajduje się lista graczy do wyboru. Po wybraniu gracza można zobaczyć posiadane przez niego gry. Poniżej gier gracza jest panel z możliwością wyboru gry do kupienia: widoczne są szczegóły wybranej gry, a także przycisk odpowiedzialny za zakup gry.

9.2 Makiety ekranów

Ekran 1 - Główny ekran administratora: wybrany ekran służy do obsługi zamówień graczy. Po lewej stronie ekranu znajduje się lista graczy. Po wybraniu gracza widoczne są posiadane przez niego gry w panelu „Player games”. Poniżej jest sekcja „Selected game details”, która pokazuje informacje o wybranej grze w panelu „Game purchase” łącznie z jej recenzjami z panelu „Reviews”. Po kliknięciu przycisku „Buy game” wybrana gra zostaje przypisana do konta aktualnie wyświetlanego gracza (o ile gracz jej już nie posiada).



10. Omówienie decyzji projektowych i skutków analizy dynamicznej

10.1 Kluczowe decyzje architektoniczne

Decyzja 1: Zastosowanie programowania obiektowego

System został zaimplementowany w języku Java z wykorzystaniem podejścia obiektowego. Każdy główny element domeny biznesowej został odwzorowany jako osobna klasa, np. Game, Player, Developer, Purchase czy Review. Pozwoliło to na czytelne odwzorowanie relacji pomiędzy obiektami oraz łatwiejsze zarządzanie logiką systemu.

Decyzja 2: Zastosowanie klasy ObjectPlus do zarządzania ekstensją

W projekcie zastosowano klasę ObjectPlus, która odpowiada za przechowywanie ekstensji obiektów. Dzięki temu możliwe jest automatyczne przechowywanie wszystkich utworzonych instancji klas dziedziczących po ObjectPlus. Rozwiązanie uprościło zarządzanie danymi oraz umożliwiło zapis i odczyt danych z pliku.

Decyzja 3: Implementacja wielodziedziczenia z wykorzystaniem interfejsów

W projekcie zaimplementowano wielodziedziczenie implementacyjne przy użyciu dziedziczenia po klasie, interfejsów oraz kompozycji. Klasa IndependentDeveloper dziedziczy po Developer, implementuje interfejs IPublisher oraz posiada obiekt klasy Publisher. Rozwiązanie to pozwoliło odwzorować niezależnego developera, który jednocześnie tworzy i wydaje własne gry.

Decyzja 4: Implementacja dynamicznego dziedziczenia

W projekcie zastosowano dynamiczne dziedziczenie dla klasy Game. Gra może zmieniać swój stan w czasie – początkowo istnieje jako GameInDevelopment, a następnie może zostać przekształcona w GameReleased.

Decyzja 5: Wykorzystanie Swing do stworzenia interfejsu użytkownika

Do implementacji interfejsu graficznego wykorzystano bibliotekę Swing. Pozwoliło to na stworzenie prostego panelu administracyjnego umożliwiającego zarządzanie danymi systemu bez konieczności korzystania z konsoli.

10.2 Decyzje dotyczące modelu danych

W systemie zastosowano relacje pomiędzy klasami odpowiadające rzeczywistym zależnościom biznesowym.

Relacja pomiędzy Player i Game została zrealizowana przy użyciu klasy asocjacyjnej Purchase, która przechowuje dodatkową informację o dacie zakupu gry.

Relacja pomiędzy Game i Review została zaimplementowana jako kompozycja, ponieważ recenzja nie może istnieć bez przypisanej gry.

W projekcie zastosowano również overlapping. Klasa Game może jednocześnie należeć do typów SINGLEPLAYER oraz MULTIPLAYER, co zostało zrealizowane przy użyciu EnumSet<GameType>.

Dane systemu są przechowywane w ekstensjach klas oraz mogą zostać zapisane do pliku i ponownie odczytane przy pomocy serializacji obiektów.

10.3 Analiza dynamiczna – wnioski

Interakcja 1: Zakup gry przez gracza

Gracz wybiera grę dostępną na platformie, a następnie system sprawdza poprawność danych oraz tworzy obiekt Purchase. Po utworzeniu zakupu gra zostaje dodana do biblioteki gracza.

Analiza wykazała, że zastosowanie klasy asocjacyjnej Purchase pozwala przechowywać dodatkowe informacje związane z zakupem bez nadmiernego komplikowania relacji pomiędzy Player i Game.

Interakcja 2: Publikacja gry przez niezależnego developera

Obiekt IndependentDeveloper może samodzielnie opublikować własną grę przy użyciu metod interfejsu IPublisher. Dzięki temu możliwe było odwzorowanie wielodziedziczenia implementacyjnego w sposób zgodny z możliwościami języka Java.

Interakcja 3: Zmiana stanu gry

Gra może przechodzić pomiędzy stanami GameInDevelopment oraz GameReleased. Zmiana stanu następuje po wywołaniu operacji odpowiedzialnej za wydanie gry.

Analiza dynamiczna wykazała, że zastosowanie dynamicznego dziedziczenia umożliwia odwzorowanie cyklu życia gry oraz zmian jej statusu w czasie działania systemu.

Interakcja 4: Dodawanie recenzji do gry

Po utworzeniu recenzji obiekt Review zostaje powiązany z konkretną grą. Dzięki zastosowaniu kompozycji recenzja nie może istnieć bez przypisanej gry.

Analiza wykazała, że takie rozwiązanie zwiększa spójność modelu danych oraz poprawnie odwzorowuje zależność biznesową pomiędzy grą a opiniami użytkowników.

Interakcja 5: Zapis i odczyt danych systemu

System przechowuje utworzone obiekty w ekstensjach klas zarządzanych przez ObjectPlus. Podczas zapisu dane są serializowane do pliku, a przy ponownym uruchomieniu aplikacji system je odczytuje.

Analiza wykazała, że mechanizm ekstensji pozwala centralnie zarządzać obiektami systemu bez konieczności tworzenia osobnej bazy danych.

10.4 Kompromisy i alternatywy

Przechowywanie listy gier bezpośrednio w klasie Player vs. Zastosowanie klasy Purchase

Alternatywą dla zastosowania klasy Purchase było przechowywanie listy gier bezpośrednio w klasie Player. Rozwiązanie to zostało odrzucone, ponieważ uniemożliwiłoby przechowywanie dodatkowych danych dotyczących zakupu, takich jak data zakupu.

Klasa Review jako niezależny obiekt vs. Kompozycja między Game i Review

Alternatywą dla zastosowania kompozycji między Game i Review było przechowywanie recenzji jako niezależnych obiektów bez silnego powiązania z grą. Rozwiązanie to zostało odrzucone, ponieważ recenzja biznesowo nie ma sensu bez gry, której dotyczy. Wybrano kompozycję, aby zapewnić spójność danych.

Kompromis: Zastosowanie ObjectPlus do zapisu ekstensji bez bazy danych

W projekcie zrezygnowano z wykorzystania bazy danych na rzecz mechanizmu ekstensji oraz serializacji obiektów do pliku. Rozwiązanie to uprościło implementację projektu i pozwoliło łatwo przechowywać relacje pomiędzy obiektami bez konieczności konfiguracji systemu bazodanowego. Kompromisem jest jednak mniejsza skalowalność oraz ograniczone możliwości wyszukiwania i zarządzania dużą liczbą danych w porównaniu do klasycznej bazy danych.

10.5 Wnioski końcowe

W ramach projektu opracowano kompletny model systemu zarządzania platformą gier komputerowych. Na etapie analizy oraz projektowania przygotowano diagramy UML, scenariusze przypadków użycia oraz projekt interfejsu użytkownika. Następnie zaimplementowano najważniejsze elementy logiki biznesowej systemu.

Projekt umożliwia zarządzanie grami, graczami, developerami, zakupami oraz recenzjami. W implementacji wykorzystano różne konstrukcje programowania obiektowego, między innymi asocjacje, klasę asocjacyjną, kompozycję, overlapping, dynamiczne dziedziczenie oraz wielodziedziczenie implementacyjne.

W projekcie zastosowano mechanizm ekstensji oraz trwałość danych opartą na serializacji obiektów do pliku zamiast klasycznej bazy danych. Rozwiązanie to uprościło implementację projektu oraz pozwoliło zachować relacje pomiędzy obiektami po ponownym uruchomieniu aplikacji.